Atividade Mockito: isolar dependências

COS482 Qualidade de Software, 2026.2. Individual, feita em casa. Entrega pelo Moodle até **sexta-feira, 09/10/2026, às 23h59**.

Objetivo

Escrever testes de unidade para uma classe que, do jeito que foi entregue, não pode ser testada: ela sorteia números, cria as próprias dependências e fala com um servidor que nunca responde. A atividade pede a refatoração que torna a classe testável e os testes, com Mockito no lugar das dependências.

O projeto

Baixe `atividade-mockito.zip` do Moodle. É um projeto Maven, com JUnit 5 e Mockito já declarados no `pom.xml`. Requer JDK 17 ou mais recente.

- **Eclipse:** File, Import, Maven, Existing Maven Projects.
- **IntelliJ IDEA e VS Code:** abrem a pasta do projeto direto.
- **Terminal:** `mvn test` na pasta do projeto.

As classes ficam no pacote `jogo`:

Classe	O que faz
`Dado`	sorteia um número de 1 a 6
`Jogador`	guarda nome e pontos; a pontuação nunca fica negativa
`ServidorDePlacar`	envia o resultado da partida para um placar online que nunca responde
`Jogo`	conduz a partida entre dois jogadores
`Principal`	joga uma partida inteira e imprime o vencedor

Rode `Principal` duas ou três vezes antes de começar. O vencedor muda a cada execução, e a mensagem de que o placar não recebeu o resultado aparece sempre. É esse comportamento que vocês vão testar.

As regras

A classe `Jogo` segue estas regras:

	Regra
R1	Em cada rodada, cada jogador lança o dado uma vez: primeiro o jogador 1, depois o jogador 2.
R2	Tirar 2, 3, 4 ou 5 soma ao jogador o valor do dado.
R3	Tirar 6 soma 11 pontos: os 6 do dado e 5 de bônus.
R4	Tirar 1 tira 3 pontos do jogador.
R5	Quem chega a 30 pontos ou mais vence, e o jogo termina na hora. Se o jogador 1 vence, o jogador 2 não lança naquela rodada.
R6	Ao vencer, o servidor de placar recebe o nome e os pontos do vencedor, uma única vez.
R7	Se o servidor estiver fora do ar, a vitória vale do mesmo jeito, o jogo termina e o registro fica pendente.
R8	`tentarRegistrarNovamente()` reenvia o resultado pendente, e a pendência some se o envio der certo. Sem pendência, lança `IllegalStateException`.
R9	Chamar `jogarRodada()` depois do fim do jogo lança `IllegalStateException`, sem lançar o dado.

A classe `Jogador` segue estas:

	Regra
J1	Começa com zero pontos e guarda o nome recebido.
J2	`adicionarPontos` soma a quantidade.
J3	`removerPontos` subtrai a quantidade, sem deixar a pontuação abaixo de zero.
J4	Nome nulo ou em branco e quantidade negativa lançam `IllegalArgumentException`.

Parte 1. Refatoração

Altere `Jogo` para que as dependências entrem pelo construtor, exatamente com esta assinatura:

```
public Jogo(Dado dado, Jogador jogador1,
            Jogador jogador2, ServidorDePlacar servidor)
```

- não mude as regras;
- não crie outros construtores nem outros métodos públicos em `Jogo`;
- ajuste `Principal` para usar o novo construtor.

Parte 2. Testes de Jogo com Mockito

Em `JogoTest`, escreva testes para as regras R1 a R9, com `Dado`, `Jogador` e `ServidorDePlacar` substituídos por mocks. Nenhum teste pode depender de sorteio nem de rede.

Parte 3. Testes de Jogador sem mock

Em `JogadorTest`, escreva testes para as regras J1 a J4 com objetos `Jogador` de verdade, sem mock.

Dicas

- O dado é lançado duas vezes na mesma rodada. `when(dado.rolar()).thenReturn(2, 4)` devolve 2 na primeira chamada e 4 na segunda.
- `registrarVitoria` não devolve nada, então não cabe dentro de `when`. Use `doThrow(...).when(servidor).registrarVitoria(...)`.
- Com `Jogador` mockado, `adicionarPontos` não altera `getPontos`. Para simular que o jogador chegou a 30, programe `getPontos` com `when`.
- Teste também o que não deve acontecer: `never()` existe para isso.
- Os slides do Mockito, da aula 13, cobrem todos os comandos necessários.

Entrega

- Rode `mvn test` e confira que tudo passa.
- Apague a pasta `target`, se ela existir.
- Compacte a pasta `atividade-mockito` inteira como `mockito_SEUDRE.zip`, por exemplo `mockito_123456789.zip`.
- Envie pelo Moodle até sexta-feira, 09/10/2026, às 23h59.

Uso de IA

Não é permitido usar assistente de IA para gerar a refatoração ou os testes. Pode usar para entender uma mensagem de erro da IDE ou do Maven.